

# Logical Operators

Truth values, precedence, and short-circuit evaluation

---

Gregory S. DeLozier, PhD

# Breaking Changes

**New keywords:** and, or, not

Rename earlier variables that use those exact names.

android, origin, and notable remain identifiers.

# Combining Comparisons

```
__input = "21";  
answer = number(input()) * 2;  
in_range = answer >= 40 and answer <= 50;  
print(in_range);
```

Two comparisons; one boolean result: **true**.

# Two Spellings, One Tag

| Source  | Token tag |
|---------|-----------|
| and, && | and       |
| or,     | or        |
| not, !  | not       |

**Normalize in the tokenizer.**

# Boolean Operations

| Left  | Right | and   | or    |
|-------|-------|-------|-------|
| false | false | false | false |
| false | true  | false | true  |
| true  | false | false | true  |
| true  | true  | true  | true  |

not true is false; not false is true.

# Precedence Changes the Tree

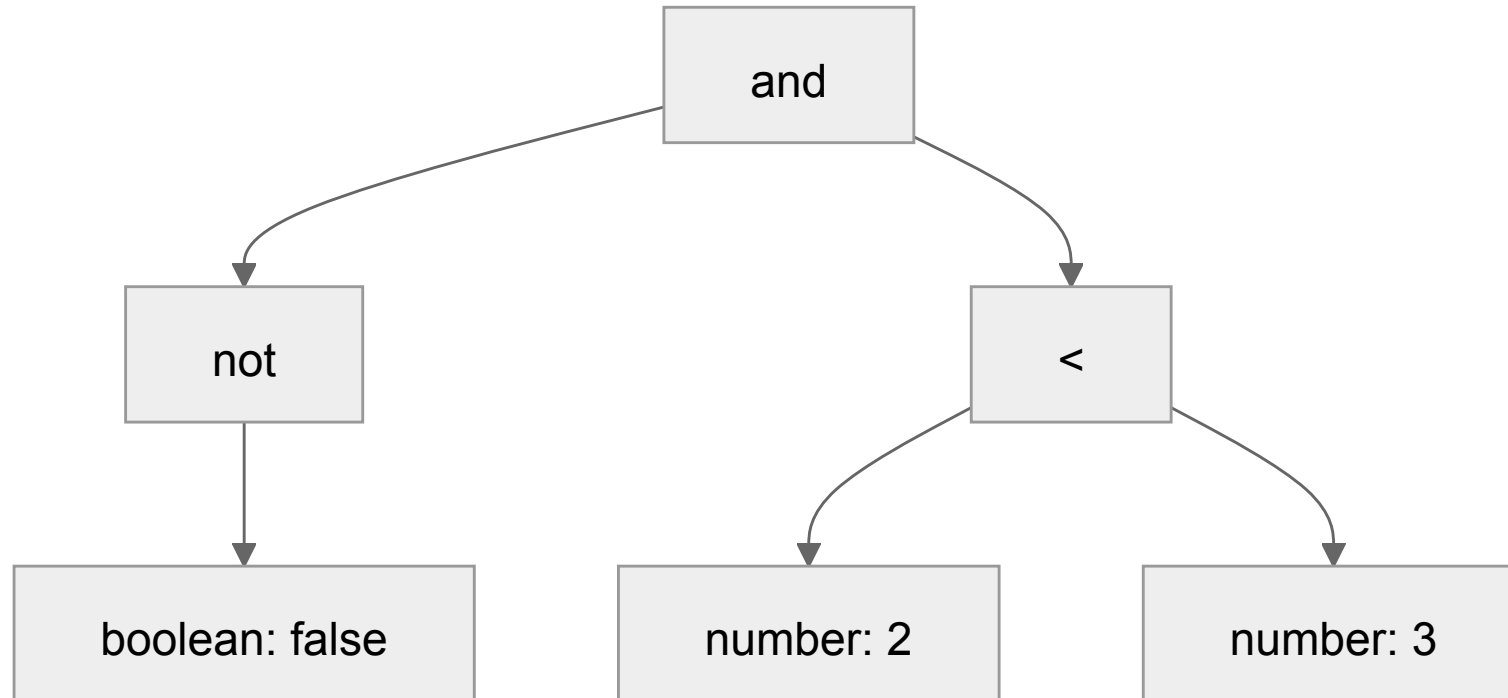
|                           |       |                             |
|---------------------------|-------|-----------------------------|
| <code>not x == y</code>   | means | <code>not (x == y)</code>   |
| <code>a or b and c</code> | means | <code>a or (b and c)</code> |

Weakest to strongest:

or, and, not, comparisons, arithmetic

# The Expression Tree

not false and 2 < 3



# Parser Levels

```
expression ::= logic_or  
logic_or ::= logic_and { "or" logic_and }  
logic_and ::= logic_not { "and" logic_not }  
logic_not ::= "not" logic_not | comparison
```

**Each level calls the next tighter level.**

# One Comparison at a Time

|                                    |                       |
|------------------------------------|-----------------------|
| <code>1 &lt; 2 &lt; 3</code>       | <code>rejected</code> |
| <code>1 &lt; 2 and 2 &lt; 3</code> | <code>true</code>     |
| <code>(1 &lt; 2) == true</code>    | <code>true</code>     |

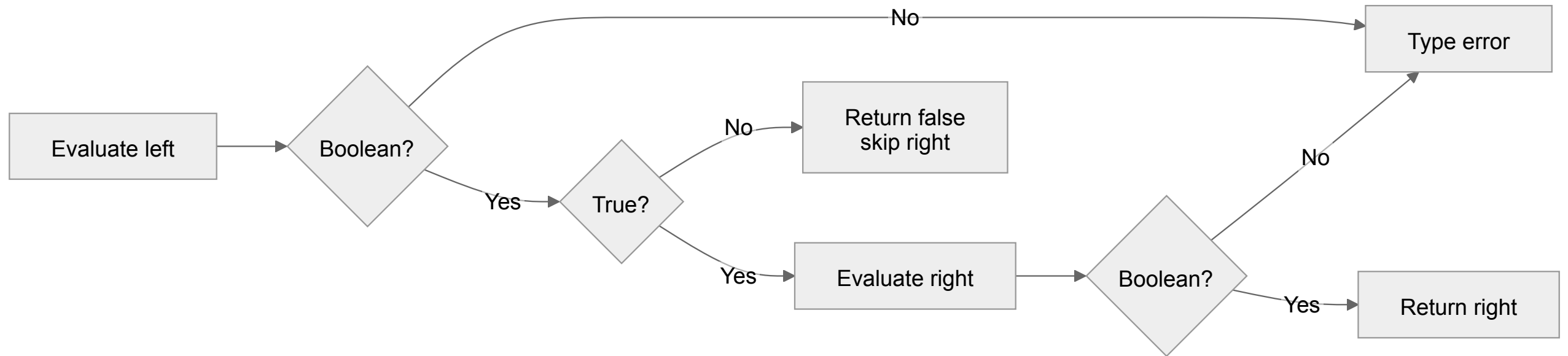
Parentheses settle grouping, not operand types.

# A Guard Before Division

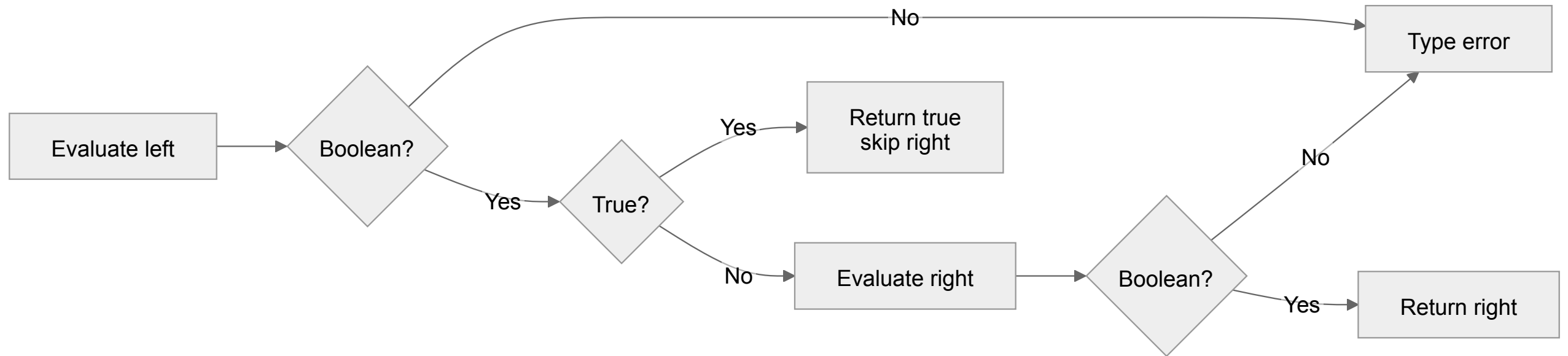
```
divisor = 0;  
safe = divisor != 0 and 10 / divisor > 2;  
print(safe);
```

**The division never runs.**

# Short-Circuit and



# Short-Circuit or



# Input That Does Not Run

```
__input = "ready";  
result = false && input() == "ready";  
print(result);  
print(__input);
```

```
false  
ready
```

# Skipped Does Not Mean Unparsed

| Expression        | Behavior     |
|-------------------|--------------|
| false and missing | false        |
| false and 42      | false        |
| true and 42       | Type error   |
| false and (1 +)   | Syntax error |

# Checking the Implementation

```
bash run-tests.sh
```

Same tokens and trees for both spellings.

Truth tables, precedence, skipped input, required input.

# Practice

Accept a supplied number from 10 through 20, inclusive.

Then compare these two expressions:

```
divisor != 0 and 10 / divisor > 2  
10 / divisor > 2 and divisor != 0
```

**When does changing the order change the behavior?**